

02 贪心算法

学习目标

- 能从题干中找到“按什么顺序处理”和“每一步选谁”的依据。
- 能读懂贪心程序中排序、扫描、状态更新、边界判断四个关键位置。
- 能完成考试风格的选择题和第 15 题风格的大题填空。
- 能结合列表、二维列表、标记数组等数据结构追踪贪心过程。

核心模型

贪心算法通常不是一次直接求出答案，而是每一步都做一个当前看来最合适的选择。考试中常见的问法不是直接问“这是不是贪心”，而是让你补全：

1. 排序依据。
2. 当前对象能否被安排。
3. 选中后状态如何更新。
4. 当前资源不够时剩余部分怎么处理。

常见代码骨架：

```
data.sort(key=lambda x: 排序依据)
初始化状态
for 当前对象 in data:
    根据当前状态判断能否安排
    如果能安排：
        更新答案
        更新状态
```

本节不再练习“从 0 写程序”，只练选择题和考试大题填空。

例题1：截止时间任务

情景题干：

某学生有若干个小任务，每个任务需要 1 天完成，有截止天数和收益。每天最多完成一个任务，任务必须安排在不晚于截止天数的某一天，要求总收益尽量高。

补全代码：

```

tasks = [{"A", 1, 30}, {"B", 2, 50}, {"C", 2, 20}, {"D", 1, 40}]
n = len(tasks)
for i in range(n - 1):
    for j in range(n - 1 - i):
        if _____①_____:
            tasks[j], tasks[j + 1] = tasks[j + 1], tasks[j]

days = [None] * 3
ans = 0
for name, deadline, value in tasks:
    for d in range(_____②_____, 0, -1):
        if days[d] == None:
            days[d] = name
            ans += _____③_____
            break

print(days, ans)

```

参考答案：

最大收益为 90，安排为第 1 天做 D，第 2 天做 B。

```

tasks[j][2] < tasks[j + 1][2]
deadline
value

```

练习1：多名学生协作任务安排

改编自第 15 题风格。

情景题干：

学校组织两名学生完成若干个志愿任务。每个任务需要 1 天完成，每天每名学生最多完成 1 个任务，因此每天最多能完成 `stu` 个任务。每个任务包含任务编号、截止天数和得分。任务必须安排在不晚于截止天数的某一天，要求总得分尽量高。为了记录安排情况，使用二维列表 `days`，其中 `days[d]` 保存第 `d` 天已安排的任务编号。

```

tasks = [
    ["A", 1, 40], ["B", 2, 60], ["C", 2, 50],
    ["D", 3, 45], ["E", 3, 30], ["F", 1, 35],
    ["G", 2, 25]
]
stu = 2

```

(1) 若按“得分从高到低，尽量放到不超过截止天数的最晚空闲日期”的策略安排，最大总得分为_____。

(2) 实现上述功能的部分 Python 程序如下，请在划线处填入合适代码。

```
n = len(tasks)
for i in range(n - 1):
    for j in range(n - 1 - i):
        if tasks[j][2] < tasks[j + 1][2]:
            tasks[j], tasks[j + 1] = tasks[j + 1], tasks[j]

maxday = 3
days = [[] for i in range(maxday + 1)]
ans = 0
for name, deadline, value in tasks:
    for d in range(____②____, 0, -1):
        if ____③____:
            ____④____
            ans += value
            break
print(days, ans)
```

(3) 若将 `days = [[] for i in range(maxday + 1)]` 改成 `days = [[]] * (maxday + 1)`，下列说法正确的是（ ）。

- A. 没有影响，两种写法完全等价
- B. 多个日期会共用同一个列表，向某一天追加任务会影响其他日期
- C. 程序无法运行，因为二维列表不能用乘法创建
- D. 只会影响 `days[0]`，不会影响第 1 天到第 3 天

参考答案：

(1) 260。B、C 放第 2 天，D、E 放第 3 天，A、F 放第 1 天，G 无空位。

(2) ① `x[2]`；② `deadline`；③ `len(days[d]) < stu`；④ `days[d].append(name)`。

(3) B。

例题2：按到货顺序装车

出处：选自 [202601-舟山-高二-期末-13](#)。

13. 有若干辆相同的货车，其最大载重为 `lim`。现有一批重量不一的货物按到货时间依次放置在一条传送带上，货物的重量按到货时间先后依次存储在列表 `a` 中，货车装载时只能按到货时

间依次取货（先到先取），现要求出装载这批货物最少需要的货车数量及各车装载情况。单件货物的重量不超过货车最大载重。

(1) 若 `a=[3,1,6,8,5,4]`，货车最大载重 `lim=10`，则最少需要 _ _ _ 辆货车。

(2) 实现上述功能的 Python 代码如下，请在划线处填入合适的代码。

```
# 获取货物重量，按到货时间先后依次记录在列表 a 中，代码略
lim = int(input("请输入货车最大载重: "))
n = _____①_____
st, cnt, tot = 0, 0, 0
for i in range(n):
    tot += a[i]
    if tot > lim:
        cnt += 1
        print("第", cnt, "辆车装载的货物编号为: ", st, "→", i - 1)
        tot = _____②_____
        st = i
    _____③_____
print("第", cnt, "辆车装载的货物编号为: ", st, "→", n - 1)
```

参考答案：

(1) 3。

(2) ① `len(a)`；② `a[i]`；③ `cnt += 1`。

练习2：贪心选择题

出处：选自 [202603-宁波十校-高三-月考-12](#)。

12. 有如下 Python 程序段：

```
# 获取 t 的值，代码略
a = [1, 2, 4, 8, 9, 11, 15, 18]
for i in range(a[len(a)-1] - a[0], -1, -1):
    tot = 1
    num = 0
    for j in range(1, len(a)):
        if a[j] - a[num] >= i:
            tot += 1
            num = j
    if tot >= t:
        break
```

执行该程序段后，i 的值为 3，则下列选项中不可能为 t 的值的是（ ）。

- A. 3
- B. 4
- C. 5
- D. 6

参考答案：

A。核验结果：t=3 时程序最终停在 i=8；t=4、t=5、t=6 时最终停在 i=3。

例题3：服务器负载分配

出处：选自 [202601-湖州-高二-期末-15](#)。

某云计算平台有 n 台服务器（编号 0 到 n-1），可用于处理若干项目的计算任务。服务器设有负载量上限，表示能同时承载的最大计算总量。每个项目包含若干具有计算量的任务。平台需将同一项目的任务合并后，合理分配给一台或多台服务器处理。分配时，优先分配给负载量最小的服务器（若负载量相同则选择编号最小的）；若分配后该服务器的负载量超出上限，则将超出部分继续分配给下一台负载量最小的服务器，直至计算量被完全分配或提示“服务器资源不足”。

项目编号	计算量
A01	40
A01	30
A02	30
B01	20
B01	60
B02	85
B02	20

第 15 题图 a

服务器编号	负载量
0	70
1	30
2	80
3	0
4	0

第 15 题图 b

平台有 5 台服务器，服务器负载量上限为 90，初始负载量均为 0，分配完项目 A01、A02、B01 的计算任务后，各服务器的负载量如第 15 题图 b 所示。

(1) 若平台有 5 台服务器，服务器负载量上限为 90，初始负载量均为 0，分配完第 15 题图 a 中项目 A01、A02、B01 的计算任务后，各服务器的负载量如第 15 题图 b 所示。则分配完所有计算任务后，服务器 4 的负载量为 _ _ _ _。

(2) 定义 merge(task) 函数，参数 task 列表每个元素表示一项计算任务，同一项目的任务连续存储。函数的功能是合并同一项目的计算任务，返回合并后计算任务的数量。

```
def merge(task):
    m, j = len(task), 0
    for i in range(m):
        if i == 0 or task[i][0] != task[i - 1][0]:
            task[j] = task[i]
            j += 1
        else:
            task[i - 1][1] += task[i][1] #加框处
    return j
```

若 task 列表依次存储上面的计算任务，调用 merge(task) 函数，请回答：

- ① 语句 task[j] = task[i] 的执行次数为 _ _ _ _。
- ② 加框处的代码有误，正确的代码应为 _ _ _ _（单选，填字母）。

- A. task[j][1] += task[i][1]
- B. task[i][1] += task[j][1]
- C. task[j-1][1] += task[i][1]

(3) 实现上述功能的部分 Python 程序如下，请在划线处填入合适的代码。

```
def arr(n, m, task):
    b = [0] * n
    f = [False] * n
    flag = False
    for i in range(m):
        if flag == True:
            break
        -----①-----
        while d > 0:
            ma = 0
            for j in range(1, n):
                if f[j] == False and b[j] < b[ma]:
                    ma = j
            if f[ma] == True:
                flag = True
                print("服务器资源不足")
                break
            elif b[ma] + d < mload:
                -----②-----
                d = 0
            else:
                d = -----③-----
```

```
b[ma] = mload
f[ma] = True
```

参考答案：

(1) 15。

(2) ① 4；② C。

(3) ① `d = task[i][1]`；② `b[ma] += d`；③ `d = d - (mload - b[ma])`。

练习3：集装箱码放

出处：选自 [202605-强基联盟-高三-月考-15](#)。

集装箱按体积从小到大分为 6 种规格，依次标记为 A-F。所有集装箱按预约时间依次到港，可竖直叠放成堆，但必须满足：后到集装箱不能放在先到集装箱的下方；体积小的集装箱不能放在体积大的集装箱下方。

为满足叠放要求，采用以下策略：新箱到港时，查找堆顶集装箱体积大于或等于新箱体积的堆。若有多个，选顶部集装箱体积最小的堆叠放；若无，则新建一堆放置。如，集装箱到港顺序为 DCBBCAABC，可以将箱子分为 [D, C, B, B, A, A]、[C, B]、[C] 三堆。设计 Python 程序，求解集装箱最少码放堆数。

(1) 如果集装箱进港顺序为 CCDBACACDBA，箱子堆数至少为 _ _ _ _。

(2) 不同时间抵达港口的集装箱，按到达时间先后依次处理。为便于后续堆放，同一时间到达的集装箱中，体积较大的先进行堆放。定义如下 `sort_d(data)` 函数，用于预处理数据，`data` 列表的每个元素含 2 个数据项，依次为到达时间、规格类型。

到达时间	规格
1	D
1	B
3	C
3	C
3	B
2	A
2	A

```
def sort_d(data):
    n = len(data)
```

```

for i in range(1, n):
    key = data[i]
    j = i - 1
    while j >= 0:
        if data[j][0] > key[0] or (data[j][0] == key[0] and data[j][1] <
key[1]):
            data[j + 1] = data[j] #加框处
            j -= 1
        else:
            break
    data[j + 1] = key
s = ""
for i in range(n):
    s = s + data[i][1]
return s

```

- ① 加框处代码的运行次数为 _ _ _ _。
- ② 在调用函数 `sort_d(data)` 后, 返回值为 _ _ _ _。

(3) 实现堆叠功能的部分程序如下, 请在划线处填入合适的代码。

```

def stacking(box):
    n = len(box)
    top = []
    top_idx = []
    prev = [-1] * n
    for i in range(n):
        cur = ord(box[i]) - 65
        left, right = 0, len(top)
        -----①-----
        while left < right:
            mid = (left + right) // 2
            if -----②-----:
                pos = mid
                right = mid
            else:
                left = mid + 1
        if pos < len(top):
            prev[i] = top_idx[pos]
            -----③-----
            top_idx[pos] = i
        else:
            top.append(cur)
            top_idx.append(i)
    return len(top)

```

参考答案：

(1) 3。

(2) ① 6；② DBAACCB。

(3) ① pos = right；② top[mid] >= cur；③ top[pos] = cur。

例题4：删除字符使数字最大

出处：选自 [202412-强基联盟-高二-月考-14](#)。

14. 有以下 Python 程序：

```
s = "4857296"
k = 3
while k > 0:
    i = 0
    while i < len(s) - 1 and s[i] >= s[i+1]:
        i += 1
    if i == len(s) - 1:
        if k > 0:
            s = s[:-k]
            k = 0
    else:
        if i == 0:
            s = s[1:]
        else:
            s = s[:i] + s[i+1:]
        k -= 1
print(s)
```

执行该程序段后，输出的值为（ ）。

- A. 8796
- B. 87296
- C. 876
- D. 8526

参考答案：

A。

课后作业

作业1：选择题

出处：选自 [202604-精诚联盟-高三-二模-13](#)。

某地电动汽车充电网络，根据各充电站实时负荷率和车辆距离，按照以下规则推荐合适的充电站：若存在充电站负荷率不高于 70% 的充电站（负荷率采用百分制，如 95% 记为 95），则选择其中距离车辆最近的；若所有充电站负荷率均高于 70%，则选择负荷率最低的；若有多个充电站满足条件，则选择编号最小的。

若某时刻编号为 0~3 的各充电站负荷率为 90, 65, 75, 50，与当前车辆距离为 2, 5, 5, 15（单位：千米），那么将会推荐的充电站编号为 _ _ _ _。

实现上述功能的 Python 程序如下，请在划线处填入合适代码。

```
"""获取所有充电站实时数据存入二维列表 d，其中 d[i][0]、d[i][1]、d[i][2] 分别存储充电站
编号、负荷系数和距离（单位：千米），且各元素均已按编号升序排序，代码略"""
mdis = 10000
id = -1
m = 0
for i in range(len(d)):
    if _____①_____:
        id = d[i][0]
        mdis = d[i][2]
        if d[i][1] < d[m][1]:
            m = i

    if _____②_____:
        print("距离最近的是", id, "号充电站")
    else:
        print("负载最低的是", _____③_____, "号充电站")
```

参考答案：推荐编号 1；① `d[i][1] <= 70 and d[i][2] < mdis`；② `id != -1`；③ `d[m][0]`。

作业2：大题填空

出处：选自 [202501-舟山-高二-期末-15](#)。

工厂在每天的生产中，需要一定数量的零件，同时知道每天生产一个零件的单价。在 n 天的生产中，当天生产的零件可以满足当天的需要，如果有多余的零件，可以放到下一天去使用，但要收取每个零件的保管费，每天收取的保管费可能不相同。

已知一个 n 天的需求量，需要制定出一个合理的生产计划，使得总费用最少。

实现该功能的 Python 程序代码如下，请在划线处填入合适的代码。

```

from random import randint
n = 100
need = [0] * n
price = [0] * n
keep = [0] * n

def make_data():
    for i in range(n):
        need[i] = randint(1, 100)
        price[i] = randint(1, 100)
        keep[i] = randint(1, 100)

def process(need, price, keep):
    p = [0] * n
    p[0] = -----①-----
    st = 0
    for i in range(1, n):
        if price[i] > price[i-1] + keep[i-1]:
            price[i] = price[i-1] + keep[i-1]
            p[st] += need[i]
        else:
            p[i] = need[i]
            -----②-----
    t = 0
    for i in range(n):
        t += -----③-----
    return p, t

-----④-----
plan, cost = process(need, price, keep)
print(plan, cost)

```

参考答案：① need[0]；② st = i；③ price[i] * need[i]；④ make_data()。

作业3：选择题

出处：选自 [202405-北斗星盟-高二-月考-15](#)。

树上有 n 个苹果，已知小明身高为 160cm，板凳高度为 40cm，每个苹果大小不一样，摘苹果和搬板凳分别需要消耗 1 个能量点。假设小明共有 ey 个能量点，如何摘能使小明摘到苹果的总重量最大？

程序思路：先按苹果高度（小于等于 160，大于 160 且小于等于 200）将数据分别存储 `apple_a`，`apple_b` 中，并按苹果重量降序排列。再对两组数据进行比较：若消耗 2 个能量点的最重苹果重量大于消耗 1 个能量点的最重两个苹果重量之和，则摘下消耗 2 个能量点的最重苹

果，否则摘下消耗 1 个能量点的最重苹果。苹果的高度与重量存储在列表 `apple` 中，每个元素中的第一个值表示苹果高度 (cm)，第二个值表示苹果重量 (g)。如 `apple = [[100,202], [210,300], [170,400], [110,100], [140,150], [180,340]]`，`ey=5`，则摘下的苹果为：`[170,400]`，`[100,202]`，`[180,340]`。请回答下列问题：

(1) 若 `apple = [[200,102], [205,200], [160,400], [150,300], [130,190], [175,104], [188,350]]`，能量点数 `ey=6`，则摘到的总重量最大是 __ g。

(2) 定义 `link(d)` 函数。函数功能是将列表 `d` 创建成链表，请在划线处填入代码。

```
def link(d):
    for i in range(len(d)):
        d[i].append(-1) # 在 d[i] 中追加一个元素 -1
    for i in range(1, len(d)):
        -----
    return d

def sort(d):
    # 列表 d 的每个元素由高度与重量构成。函数功能是将 d 按重量降序排列，代码略
```

(3) 实现摘苹果的主程序如下，请在划线处补充代码。

```
apple = [[100,202],[210,300],[170,400],[110,100],[140,150],[180,340]]
apple_a = []; apple_b = []
# 按苹果高度（小于等于 160，大于 160 且小于等于 200）将数据分别存储在 apple_a, apple_b 中
# 代码略
sort(apple_a); sort(apple_b); link(apple_a); link(apple_b)
head_a, head_b = 0, 0; apple_end = []
alen = len(apple_a); blen = len(apple_b)
ey = int(input("请输入能量值:"))

def linkdel(d, head, dlen):
    apple_end.append(d[head])
    -----①-----
    dlen = dlen - 1
    return head, dlen

while ey > 0:
    if alen > 1 and blen > 0:
        if -----②----- and ey >= 2:
            ey = ey - 2
            head_b, blen = linkdel(apple_b, head_b, blen)
        else:
            ey = ey - 1
```

```

        head_a, alen = linkdel(apple_a, head_a, alen)
    elif blen == 0 and alen > 0:
        ey = ey - 1
        head_a, alen = linkdel(apple_a, head_a, alen)
    elif blen > 0 and alen == 1:
        if ey > 1 and apple_b[head_b][1] > apple_a[head_a][1]:
            ey = ey - 2
            head_b, blen = linkdel(apple_b, head_b, blen)
        else:
            ey = ey - 1
            head_a, alen = linkdel(apple_a, head_a, alen)
    elif blen > 0 or alen == 0:    #加框处
        ey = ey - 2
        head_b, blen = linkdel(apple_b, head_b, blen)
    else:
        break
print("剩下能量值:", ey)
print("摘下的苹果:", end="")
for i in range(len(apple_end)):
    print(apple_end[i][:2], end="")

```

(4) 程序中加框处有误，请修正。

参考答案： (1) 1240; (2) `d[i-1][2]=i`; (3) ① `head=d[head][2]`; ② `apple_b[head_b][1]>apple_a[head_a][1]+apple_a[apple_a[head_a][2]][1]`; (4) `blen>0 and alen==0 and ey>1`

作业4：大题填空

出处：选自 [202501-衢州-高二-期末-15](#)。

某智能工厂有编号为 0 至 $n-1$ 的 n 个生产车间，各车间每生产一件产品，其信息（产品编号、起点车间、终点车间及重量）自动上传至工厂系统，之后等待载重为 m 的无人送货车往返运输至终点车间继续加工。无人送货车先后经过产品的起点车间和终点车间，且装货后不超重，则运输该产品。如第15题图a所示的G01产品，若某次车辆从0车间行驶到 $n-1$ 车间，且装货后不超重，则运输该产品。小明根据产品数据，编写程序模拟运输过程，输出每次往返运输的产品编号。



第 15 题图 a

定义如下 `classify(data)` 函数，参数 `data` 中每个元素由产品编号、起点车间、终点车间、重量及状态 5 项构成。函数功能是将元素分类存储，列表 `pos` 存储起点车间编号小于终点车间编号的产品数据并按起点车间编号升序排列，列表 `opp` 存储起点车间编号大于终点车间编号的产品数据并按起点车间编号降序排列。

```
def classify(data):
    pos = []
    opp = []
    for i in range(len(data)):
        if _____①_____:
            pos.append(data[i])
        else:
            opp.append(data[i])
    return pos, opp

def arrange(lst, flag, n, m):
    weight = [0] * n
    bh = []
    num = 0
    for i in range(len(lst)):
        if lst[i][4] == False and _____②_____:
            lst[i][4] = True
            num += 1
            bh.append(lst[i][0])
            if flag == True:
                for j in range(lst[i][1], lst[i][2]):
                    weight[j] += lst[i][3]
            else:
                for j in range(_____③_____):
                    weight[j] += lst[i][3]
    return bh, num
```

'''

所有产品数据存入列表`data`，`data`中每个元素由产品编号、起点车间、终点车间及重量4项构成，例如`[[C01,2,8,7],[E01,4,15,10],[S01,18,5,12],[G01,6,11,6]]`
读取车间数量、车辆载重量分别存入变量`n`和`m`中

'''

```
for i in range(len(data)):
    data[i].append(False)
pos, opp = classify(data)
count = 0
while count < len(data):
    bh1, num1 = arrange(pos, True, n, m)
    bh2, num2 = arrange(opp, False, n, m)
    count += _____④_____
```

参考答案：① `data[i][1] < data[i][2]`；② `lst[i][3] <= m - weight[lst[i][1]]`；③ `lst[i][2]+1, lst[i][1]+1`；④ `num1 + num2`。

作业5：大题填空

出处：选自 [202505-北斗星盟-高二-月考-15](#)。

某快递站接收到的包裹有优先包裹和普通包裹 2 种类型，快递站将它们依次分拣到 $n \times n$ 的货架上。优先包裹存放在货架的下半区，普通包裹存放在货架的上半区。快递站将包裹分拣完成后，还需要对相同类型且送往相同小区的包裹进行装箱处理，以减少派送次数。

装箱规则：箱子的最大载重量为 `maxw`，为使装箱后箱子数量尽可能少，采用最佳匹配算法进行装箱，即选择当前箱子剩余容量与待放入包裹重量最接近的箱体进行装载。

实现题目功能的部分 Python 程序如下，请在划线处填入合适的代码。

```
def fz(data, m, n):
    heads = [[-1] * m for i in range(2)]
    for i in range(len(data)):
        t = int(____①____)
        k = data[i][3] - 1
        if heads[t][k] == -1:
            heads[t][k] = i
        else:
            data[i][4] = heads[t][k]
            heads[t][k] = i
    return data, heads

def cal_groups(data, head, maxw):
    re = []
    p = head
    while p != -1:
        w = data[p][2]
        k = data[p][4]
        min_remain = maxw
        idx = -1
        for i in range(len(re)):
            if ____②____:
                idx = i
                min_remain = re[i][0]
        if idx != -1:
            re[idx][0] -= w
            pre_tail = re[idx][2]
            data[pre_tail][4] = p
            data[p][4] = -1
            re[idx][2] = p
```

```

else:
    data[p][4] = -1
    re.append([maxw - w, p, p])
    -----③-----
return re

```

参考答案: ① `data[i][1] // (n // 2)`; ② `re[i][0] >= w and re[i][0] < min_remain`;
 ③ `p = k`。

作业6：大题填空

出处：选自 [202605-上虞-高三-月考-15](#)。

某工厂车间拥有多条可同时运行的生产流水线，现有若干零件加工生产任务。每个加工任务均有固定的开始加工时刻、结束加工时刻，且单个任务全程需要同时占用若干条流水线，加工过程不可中断、不可拆分。同一时间段相互重叠的加工任务，所占用流水线不可共用。现开发一个系统，用于统计高峰期至少同时需要几条流水线，以及高峰期所用流水线累计时间。

编号	开始加工时刻	结束加工时刻	占用流水线数量
1	9:00	10:55	2
2	9:30	11:20	1
3	10:40	12:00	2
4	11:00	12:50	2

例如有 4 个零件要加工（如上表），零件的情况可以表示为 `[["9:00", "10:55", 2], ["9:30", "11:20", 1], ["10:40", "12:00", 2], ["11:00", "12:50", 2]]`（其中每个三元组分别表示零件开始加工时刻、结束加工时刻以及占用流水线数量），工厂高峰期 10:40 至 10:55 和 11:00 至 11:20 至少同时需要使用 5 条流水线，累计用时 35 分钟。现开发一个系统，用于统计高峰期至少同时需要几条流水线，以及高峰期所用流水线累计时间。

(1) 定义如下 `sort_data(a)` 函数，参数 `a` 的每个元素由时间（已经转化为分钟）、占用流水线数量以及状态（0 表示开始，1 表示结束）3 个数据项组成。函数的功能是根据时间大小，对 `a` 进行升序排序。

```

def sort_data(a):
    n = len(a)
    for i in range(n - 1):
        flag = 0
        ##虚线框开始
        for j in range(n - 1, i, -1):
            if a[j][0] < a[j - 1][0]:
                a[j], a[j - 1] = a[j - 1], a[j]

```

```

        flag = 1
    if flag == 0:
        break
    ##虚线框结束
return

```

调用该函数，若 a 为 $[[540, 2, 0], [655, 2, 1], [570, 1, 0], [680, 1, 1], [640, 2, 0], [660, 2, 1]]$ ，请回答下列两个问题。

① 虚线框中的程序段第 1 次执行后，下列说法正确的是 __（单选）。

- A. $a[0][0]$ 数值最小
- B. $a[0][0]$ 数值最大
- C. $a[5][0]$ 数值最小
- D. $a[5][0]$ 数值最大

② 虚线框中的程序段执行的次数为 __。

(2) 实现上述功能的部分 Python 程序如下：

请在划线处填入合适的代码。

```

def st(tim):
    # 函数功能：将时间 tim 转化为分钟形式，例如 "9:30" 转化为 570
    w = ""
    for c in tim:
        if c == ":":
            h = int(w)
            w = ""
        else:
            w = w + c
    m = int(w)
    return -----①-----

"""
data[i] 包含 3 项，data[i][0] 存放零件开始加工时刻，
data[i][1] 存放零件结束加工时刻，data[i][2] 存放占用流水线数量
"""
a = []
for i in range(len(data)):
    d = data[i]
    a.append([st(d[0]), d[2], 0]) # 为 a 追加一个元素
    a.append([st(d[1]), d[2], 1])
sort_data(a)
i, s, maxv, t = 0, 0, 0, 0
while i < len(a):

```

```

j = i
while j < len(a) and a[i][0] == a[j][0]:
    if a[j][2] == 0:
        s += a[j][1]
    else:
        -----②-----
        j += 1
if j == len(a):
    break
atime = -----③-----
if s > maxv:
    maxv = s
    t = atime
elif s == maxv:
    t += atime
i = j
print("至少需要", maxv, "条流水线, 占用", t, "时间")

```

参考答案： (1) ① A; ② 3; (2) ① $h*60+m$; ② $s-=a[j][1]$; ③ $a[j][0]-a[j-1][0]$ 或 $a[j][0]-a[i][0]$

内层 while 循环通过 $j += 1$ 不断向后扫描, 直到 $a[j][0] != a[i][0]$ 时才会退出。也就是说, 退出循环时, $a[i][0]$ 代表的是**当前处理的时刻**, 而 $a[j][0]$ 正好代表**下一个发生变化的时刻**。

错题复盘栏

错题	错因	正确贪心依据

教师观察记录

观察点	表现
是否能找排序依据	
是否能更新状态	
是否能追踪程序输出	
是否能说明边界条件	